

Project 3

Due Date: Monday 29 July 2024 by 8:00 AM

General Guidelines.

The instructions given below describe the required methods in each class. You may need to write additional methods or classes that will not be directly tested but may be necessary to complete the assignment.

In general, you are not allowed to import or use any additional classes in your code without explicit permission from your instructor!

Note: It is okay to use the `Math` class if you want.

Project Overview

In this project, you will write some sorting algorithms for different array input, trying to get good runtimes for the various types of inputs. You will also implement two versions of a “sort” for a grid, again trying to get good runtimes. Finally, you will implement a Trie data structure.

Part 1. ArraySort

In this part, you will be implementing sorting algorithms for Arrays based on what you know about the input array. In each case, you are given specific information about the input array and you should try to write an algorithm that will work well for that specific array. Note that you are not required to write different sorting algorithms for each of the 7 input arrays, but using the same algorithm for all 7 inputs is unlikely to receive full credit.

Note: All the sorting methods should put the array in ascending order.

Note: You must use the `Array` class that is provided and you must be sure not to get around the access counts in any way. That means that anything you do that lowers the access counts should actually be improving your runtime, not just avoiding access counts.

Here are the seven inputs you need to consider.

1. The input array is in descending order.
2. The input array is in random order.
3. The input array is almost sorted, meaning that only a few items are out of order.
4. The input array is locality-aware. That means that every item in the input array is no more than d positions away from where it will end up in the sorted array.

5. The input array is mostly sorted with just a small number of unsorted elements at the end.
6. The input array is made up of a small number of sorted subarrays.
7. The input array is random, but the values in the array are relatively limited in range.

Implementation.

- Implement your solution in a file called *ArraySort.java*. All of the test cases will call a single method, whose signature is given below.

```
public static void sort(Array array, int num, int d)
```

- array: the input array
 - num: the input number (1-7)--see above
 - d: the d value for #4--all other cases will ignore this
- Specific guidelines:
 - In general, you should be doing your sorting using the Array class. Any external data structures should be limited to $O(1)$ space and mostly $O(1)$ time. The exceptions are:
 - You may use the extra array provided through the Array.java class, which is particularly important if you are doing any version of Mergesort.
 - You may use an ArrayList in #6 but only to keep track of the indexes of the sorted subarrays.
- I recommend that you have separate methods for each of your sorting algorithms and just call those inside the main sorting algorithm according to what the *num* argument is.

Testing.

- Test cases and test code is provided for you. Keep in mind that these may not necessarily catch all possible errors, so it's up to you to make sure you are handling any corner cases.
- The test cases use specific inputs for each scenario (which you can see in the text files). These are quite large because we want to get an accurate idea of your runtime.
- For each test case, the code checks accuracy (i.e. is it sorted?) and efficiency (by checking access counts) and awards points based on those results. If your algorithm is not very efficient, you may not receive full credit but there are levels of efficiency.

- The total for this part of the project is 30 points, but full credit in the auto-grader is actually 31.5 points. The extra 1.5 points are extra credit.
- The test code should give you a reasonable idea of what you can expect from the grading code and tests.
- Note that if you use any randomization in your algorithm (e.g. Quicksort) your access counts may vary somewhat. If you don't get the grade you expect because of this, we can always run your code a few extra times to see if it improves, you will have to let us know during the regrade window. Also note that importing the Random class is okay for this purpose.

Provided Code.

Besides the test code, you are provided with the *Array.java* class. You should NOT change this code in any way that will affect your code's ability to work as it will not be included in your submission. However, feel free to change it for testing purposes.

Part 2. SortGrid

In this case, you are going to “sort” a grid in two different ways based on two different definitions of a “sorted” grid.

Definition 1: A sorted grid is one where, if you laid the rows side by side, the resulting array would be sorted.

Example:

-3	-2	-2	0	4
4	4	5	5	6
7	8	9	9	9
10	11	12	12	15
16	20	22	23	25

Definition 2: A sorted grid is one where each element is less than or equal to the elements directly to the right and directly down from itself.

Example:

0	0	2	3	3
1	2	4	4	5
3	3	5	5	6

4	4	5	6	7
5	5	5	6	8

A small hint for Definition 2: It tends to surprise students how simple a solution to this can be. :-)

Implementation.

- Implement your solution in a file called *SortGrid.java*. Two methods are required and their signatures are given below.

```
public static void sortA(Grid g)
```

- *g*: the Grid to be sorted
- result: *g* should be sorted according to Definition 1 above

```
public static void sortB(Grid g)
```

- *g*: the Grid to be sorted
- result: *g* should be sorted according to Definition 2 above

- Specific guidelines:
 - In general, you should be doing your sorting using the Grid class.
 - The *Grid.java* class does have a mechanism for providing an extra grid that you can access in a way that will still count toward your overall count. For example, you might want to use this if you are doing some kind of Mergesort algorithm. But of course, this limits you to just one extra grid and accessing it does affect your overall access count, so use it wisely.
 - I recommend that you think about the Grid as a grid and not as an array.
 - You may not use any additional data structures for this part.

Testing.

- Test cases and test code are provided for you. Keep in mind that these may not necessarily catch all possible errors, so it's up to you to make sure you are handling any corner cases.
- There are five input grids that will each be run with *sortA* and *sortB*.
- For each test case, the code checks accuracy (i.e. is it sorted?) and efficiency (by checking access counts) and awards points based on those results. If your algorithm is not very efficient, you may not receive full credit but there are levels of efficiency.
- The total points for this part is 20 points, but the auto-grader goes up to 22. The extra 2 points are extra credit.

- The test code should give you a reasonable idea of what you can expect from the grading code and tests.

Provided Code.

Besides the test code, you are provided with the *Grid.java* and *Loc.java*. You should NOT change this code in any way that will affect your code's ability to work as it will not be included in your submission. However, feel free to change it for testing purposes. Note: It is important to note that these grids are all made up of integers, so you will want to use the `getIntVal` and `setIntVal` methods instead of `getVal` and `setVal`.

Part 3. Trie

On HW 8, you were introduced to a tree structure used for searching that is called a Trie. This is a structure that is useful for searching for strings, typically in a fixed text since building the Trie takes time. In this part, you will finish an implementation of a Trie. Starter code is provided for you in the class called `Trie.java`, and you should complete the code there. Additionally, a set of slides is provided for you to give you more information about Tries. Note that you are only expected to implement the standard Trie. Specific information for the required methods are given in the starter code.

To test the Trie, I have also provided a class called `TrieRun.java`, which is a main method that allows you to input a file to build a Trie, then search the Trie. I have also provided a very small file (`input.txt`) and a sample interaction for your reference, but you are strongly encouraged to test your code more extensively on larger inputs. Note that indexing for lines and words should start at 1.

As usual, you are not allowed to import any additional structures or use any additional structures other than what is already in the starter code.

Grading.

To test your code, we will run the `TrieRun.java` to test the functionality. This part of the project is worth 30 points, and the points will be roughly divided between searching for full words and searching by prefix. However, as usual, we will also check your code by hand.

Sample Interaction with `TrieRun.java`.

Enter a file name:

`input.txt`

Type "w" to search for a specific word.

Type "p" to search for words by a specific prefix.

```
Type "q" to quit.
w
Type the word:
seashore
Line 3, Word 3
Line 6, Word 2
Line 9, Word 2
w
Type the word:
sea
That word is not in the document.
p
Type the prefix:
sea
seashells
seashore
p
Type the prefix:
sal
That prefix is not in the document.
q
```

Submitting, Testing, and Grading

Your code must compile and run on lectura, and it is up to you to check this before submitting.

To test your code on lectura:

- Transfer all the files (including test files) to lectura.
- Run `javac *.java` to compile all the java files. (You can also use `javac` with an individual java file to compile just the one file: `javac ArrList.java`)
- Run `java <filename>` to run a specific java file. (Example: `java ArrListTest.java`)

After you are confident that your code compiles and runs properly on lectura, you can submit the required files using the following **turnin** command. Please do not submit any other files than the ones that are listed.

```
turnin csc345pa3 ArraySort.java SortGrid.java Trie.java
```

Upon successful submission, you will see this message:

```
Turning in:
ArraySort.java -- ok
```

SortGrid.java -- ok
Trie.java
All done.

Note: Only properly submitted projects are graded! No projects will be accepted by email or in any other way except as described in this handout and the syllabus. If you are worried about your submission, feel free to ask us to check that it got into the right folder.

Grading.

Your score will be determined by the tests we do on your code (which are similar to the provided tests) minus any deductions that are applied when your code is manually graded. In addition to late deductions, coding style, and deductions for not following directions, you may also receive deductions for inefficient code.

For Coding Style, we look for:

- understandable code
- helpful comments
- good indentation
- good method abstraction—i.e. you should be writing reusable methods rather than repeating the same code multiple times

See the resubmission policy in the syllabus.

It is absolutely not okay to ask ChatGPT to give you a solution to this problem, nor is it okay for you to get solutions from any other sources or to share your solution with anyone else.